

Exercise 1: Implementing the Singleton Pattern

C# Code:

```
using System;
// Define Singleton Class
public class Logger
{
    private static Logger instance;
    private static readonly object lockObj = new object();
    private Logger()
    {
        Console.WriteLine("Logger initialized.");
    }
    // Implement Singleton Pattern
    public static Logger GetInstance()
    {
        if (instance == null)
        {
            lock (lockObj)
            {
                if (instance == null)
                {
                    instance = new Logger();
                }
            }
        }
        return instance;
    }
    public void Log(string message)
    {
        Console.WriteLine("LOG: " + message);
    }
}
// Test Singleton Implementation
class Program
{
    static void Main(string[] args)
    {
        Logger logger1 = Logger.GetInstance();
        Logger logger2 = Logger.GetInstance();
        logger1.Log("First log message.");
        logger2.Log("Second log message.");
        if (ReferenceEquals(logger1, logger2))
        {
            Console.WriteLine("Both logger instances are the same!");
        }
        else
        {
            Console.WriteLine("Different instances exist!");
        }
    }
}
```

Output:

```
Logger initialized.  
LOG: First log message.  
LOG: Second log message.  
Both logger instances are the same!
```

Exercise 2: Implementing the Factory Method Pattern

C# Code:

```
using System;  
// Define Document Interface  
public interface IDocument  
{  
    void Open();  
}  
// Create Concrete Document Classes  
public class WordDocument : IDocument  
{  
    public void Open()  
    {  
        Console.WriteLine("Opening Word Document...");  
    }  
}  
public class PdfDocument : IDocument  
{  
    public void Open()  
    {  
        Console.WriteLine("Opening PDF Document...");  
    }  
}  
public class ExcelDocument : IDocument  
{  
    public void Open()  
    {  
        Console.WriteLine("Opening Excel Document...");  
    }  
}  
// Implement Factory Method  
public abstract class DocumentFactory  
{  
    public abstract IDocument CreateDocument();  
}  
public class WordDocumentFactory : DocumentFactory  
{  
    public override IDocument CreateDocument()  
    {  
        return new WordDocument();  
    }  
}  
public class PdfDocumentFactory : DocumentFactory  
{  
    public override IDocument CreateDocument()
```

```

    {
        return new PdfDocument();
    }
}
public class ExcelDocumentFactory : DocumentFactory
{
    public override IDocument CreateDocument()
    {
        return new ExcelDocument();
    }
}
// Test Factory Method Implementation
class Program
{
    static void Main(string[] args)
    {
        DocumentFactory wordFactory = new WordDocumentFactory();
        IDocument wordDoc = wordFactory.CreateDocument();
        wordDoc.Open();
        DocumentFactory pdfFactory = new PdfDocumentFactory();
        IDocument pdfDoc = pdfFactory.CreateDocument();
        pdfDoc.Open();
        DocumentFactory excelFactory = new ExcelDocumentFactory();
        IDocument excelDoc = excelFactory.CreateDocument();
        excelDoc.Open();
    }
}

```

Output:

```

Opening Word Document...
Opening PDF Document...
Opening Excel Document...

```